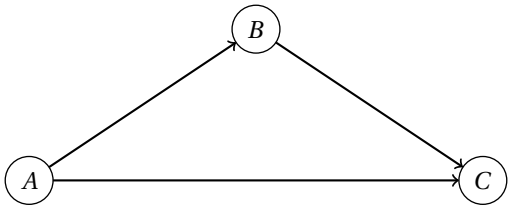
Bayesian Networks (BN)

1 Definition

Definition 1.1 (Bayesian Network)**.**
A **Bayesian Network (BN)** is a probabilistic graphical model that represents a set of variables and their conditional dependencies.

- It is a **directed acyclic graph (DAG)**.
- Every **node** represents a **random variable** and is associated with a **conditional probability table (CPT)**.
- The **Local Markov Property** holds: each random variable is conditionally independent of its non-descendants given its parents.

An example



Conditional Probability Tables (CPTs)

A	$P(A)$
1	0.2
2	0.5
3	0.3

A	B	$P(B \mid A)$
1	T	0.7
1	F	0.3
2	T	0.4
2	F	0.6
3	T	0.9
3	F	0.1

A	1	1	1	1	2	2	2	2	3	3	3	3
B	T	T	F	F	T	T	F	F	T	T	F	F
C	+1	−1	+1	−1	+1	−1	+1	−1	+1	−1	+1	−1
$P(C \mid A, B)$	0.8	0.2	0.5	0.5	0.6	0.4	0.3	0.7	0.9	0.1	0.4	0.6

1.1 The CPTs

For a node without parents, the CPT simply gives the prior distribution of the variable.

For a random variable X_i with parents $\text{Parents}(X_i)$, the conditional probability table (CPT) specifies the probabilities $P(X_i \mid \text{Parents}(X_i))$.

To be more specific, let X be a node in the BN and let the set of its parents $\text{Parents}(X) = \{Y_1, Y_2, \ldots, Y_n\}$. Suppose:

- $\boldsymbol{Y} = (Y_1, Y_2, \ldots, Y_n)$ is a tuple of the **parents** of X.
- $\boldsymbol{y} = (y_1, y_2, \ldots, y_n)$ denotes a variable that takes values of $\boldsymbol{Y}$.

The CPT of node X defines a multi-variable function as follows:

$$\begin{aligned} f(x, \boldsymbol{y}) &:= P(X = x \mid \boldsymbol{Y} = \boldsymbol{y}) \\ &= P(X = x \mid \{Y_1 = y_1, Y_2 = y_2, \ldots, Y_n = y_n\}) \\ &= P(X = x \mid \bigcup_{Y_j \in \text{Parents}(X)} \{Y_j = y_j\}). \end{aligned}$$

1.2 The Local Markov Property

Definition 1.2 (Conditional Independence)**.**
Let A, B, and C be three events. A is said to be **conditionally independent** of B given C, denoted as $(A \perp B) \mid C$, if and only if:

$$P(A \mid B \cup C) = P(A \mid C),$$

or equivalently,

$$P(A \cap B \mid C) = P(A \mid C) \cdot P(B \mid C)$$

The Local Markov Property states that a node is conditionally independent of its non-descendants given its parents. Formally, for a node X in the BN:

$$X \perp \text{NonDescendants}(X) \mid \text{Parents}(X),$$

or more accurately,

$$X \perp \text{NonDescendants}(X) \setminus \text{Parents}(X) \mid \text{Parents}(X).$$

This leads to the following equation:

$$P\big(X \mid \text{Parents}(X) \cup \text{NonDescendants}(X)\big) = P\big(X \mid \text{Parents}(X)\big)$$

2 Properties

Theorem 2.1 (Factorization Theorem / Chain Rule)**.**
Given a Bayesian Network with nodes $\boldsymbol{U} = \{X_1, X_2, \ldots, X_n\}$, the joint probability distribution of all random variables can be factorized as:

$$P(X_1 = x_1, X_2 = x_2, \ldots, X_n = x_n) = \prod_{X_i \in \boldsymbol{U}} P\Big(X_i = x_i \mid \bigcup_{X_j \in \text{Parents}(X_i)} \{X_j = x_j\}\Big)$$

Given the values $x_1, x_2, \ldots, x_n$, every factor on the RHS can be obtained from the CPT of a node.

***Proof*.**
The proof is based on three preliminary points:

1. General Chain Rule For any random variables $X_1, X_2, \ldots, X_n$, the joint probability distribution can be expressed as:

$$\begin{aligned} P(X_1, X_2, \ldots, X_n) &= P(X_1) \cdot P(X_2 \mid X_1) \cdot P(X_3 \mid \{X_1, X_2\}) \cdots P(X_n \mid \{X_1, X_2, \ldots, X_{n-1}\}) \\ &= \prod_{i=1}^{n} P\Big(X_i \mid \{X_1, X_2, \ldots, X_{i-1}\}\Big) \\ &= \prod_{i=1}^{n} P\Big(X_i \mid \bigcup_{j=1}^{i-1} \{X_j = x_j\}\Big) \end{aligned}$$

2. The Local Markov Property Let X be any node in the BN, $\text{NonDescendants}(X)$ be the set of its non-descendants, and $\text{Parents}(X)$ be the set of its parents. $\text{Parents}(X) \subseteq \text{NonDescendants}(X)$. The property gives:

$$P\big(X \mid \text{NonDescendants}(X)\big) = P\big(X \mid \text{Parents}(X)\big)$$

3. Topological Ordering A DAG has at least one topological ordering, which means we can find an arrangement $X_1, X_2, \ldots, X_n$ such that for any X_i, $\text{Parents}(X_i) \subseteq \{X_1, X_2, \ldots, X_{i-1}\}$.

Based on the above three points, let $X_1, X_2, \ldots, X_n$ be in topological order, then we have:

$$\begin{aligned} P(X_1, X_2, \ldots, X_n) &= \prod_{i=1}^{n} P\Big(X_i \mid \{X_1, X_2, \ldots, X_{i-1}\}\Big) \\ &= \prod_{i=1}^{n} P\Big(X_i \mid \text{NonDescendants}(X_i)\Big) \\ &= \prod_{i=1}^{n} P\Big(X_i \mid \text{Parents}(X_i)\Big) \end{aligned}$$

□

3 Inference in a Bayesian Network

Some basic concepts:

- **Query**: The variables we want to know about their distributions.
- **Evidence**: The variables and their observed values.

- **Hidden Variables**: The variables that are neither query nor evidence.
- **Goal**: Compute the conditional probability distribution of the query variables given the evidence.

Different algorithms can be used for inference in Bayesian Networks, including:

- **Exact Inference**: Enumeration, Variable Elimination, Belief Propagation.
- **Approximate Inference**: Monte Carlo methods, Variational Inference.

3.1 Inference by Enumeration

3.1.1 The Method

- Query variable: X
- The tuple of evidence variables: $\boldsymbol{E} = (E_1, E_2, \ldots, E_m)$
- The tuple of observed values for evidence variables: $\boldsymbol{e} = (e_1, e_2, \ldots, e_m)$
- The tuple of hidden variables: $\boldsymbol{H} = (H_1, H_2, \ldots, H_k)$
- The tuple of values for hidden variables: $\boldsymbol{h} = (h_1, h_2, \ldots, h_k)$

$$\begin{aligned} P(X = x \mid \boldsymbol{E} = \boldsymbol{e}) &= \frac{P(X = x, \boldsymbol{E} = \boldsymbol{e})}{P(\boldsymbol{E} = \boldsymbol{e})} \\ &= \alpha \cdot P(X = x, \boldsymbol{E} = \boldsymbol{e}) \\ &= \alpha \cdot \sum_{\boldsymbol{h}} P(X = x, \boldsymbol{H} = \boldsymbol{h}, \boldsymbol{E} = \boldsymbol{e}) \\ &= \alpha \cdot \sum_{h_1} \cdots \sum_{h_k} P(X = x, H_1 = h_1, \ldots, H_k = h_k, \boldsymbol{E} = \boldsymbol{e}) \end{aligned}$$

where α is the normalization constant that can be found after computing $P(X = x, \boldsymbol{E} = \boldsymbol{e})$ for all values of x.
The term $P(X = x, \boldsymbol{H} = \boldsymbol{h}, \boldsymbol{E} = \boldsymbol{e})$ is a joint probability distribution and can be computed using the Factorization Theorem.

3.1.2 Time Complexity

Suppose:

- The BN has n nodes in total.
- Each random variable has at most d possible values.
- There are k hidden variables.

Further suppose reading from a CPT is $O(1)$, then computing each $P(X = x, \boldsymbol{H} = \boldsymbol{h}, \boldsymbol{E} = \boldsymbol{e})$ takes $O(n)$.

Conclusion The final time complexity is $O(n \cdot d^k)$, or $O(n \cdot d^n)$ as the evidence variables are usually only a few.

3.2 Variable Elimination

3.2.1 Some Definitions

1. Factors A **factor** is a multi-variable function that maps some values taken by a set of variables to a real number. For example, a factor $f(X, Y, Z)$ can be defined as:

$$f(x, y, z) = P(X = x \mid Y = y, Z = z)$$

CPTs are also factors.

The order of parameters of a factor does not matter. A variable is dependent on all other variables in the factors where it appears. The dependences among variables are not directed.

Scope of a factor The **scope** of a factor is the set of variables that the factor depends on. For example, the scope of factor $f(X, Y, Z)$ is $\{X, Y, Z\}$.

2. Pointwise product The **pointwise product** of two factors yields a new factor, whose scope is the union of the scopes of the two factors. Let $f_1(X_1, X_2, \ldots, X_n, Z_1, Z_2, \ldots, Z_k)$ and $f_2(Y_1, Y_2, \ldots, Y_m, Z_1, Z_2, \ldots, Z_k)$ be two factors with common variables $Z_1, Z_2, \ldots, Z_k$. The pointwise product $f = f_1 \cdot f_2$ is defined as:

$$f(x_1, x_2, \ldots, x_n, y_1, y_2, \ldots, y_m, z_1, z_2, \ldots, z_k) := f_1(x_1, x_2, \ldots, x_n, z_1, z_2, \ldots, z_k) \cdot f_2(y_1, y_2, \ldots, y_m, z_1, z_2, \ldots, z_k)$$

3. Summing out Factors show the dependences among variables. When a variable only appears in one factor, we can conclude that it only depends on the variables in that factor. Thus, we can **sum up** all possible values of that variable to eliminate it from the factor.

Let $f(X, Y_1, Y_2, \ldots, Y_n)$ be a factor. The operation of **summing out** variable X from factor f yields a new factor ϕ defined as:

$$\phi(y_1, y_2, \ldots, y_n) := \sum_x f(x, y_1, y_2, \ldots, y_n)$$

3.2.2 Algorithm Steps

Throughout the algorithm, we maintain a set of factors.

1. The initial set of factors The initial factors are the CPTs of all nodes in the BN.

Evidence Instantiation If a factor contains evidence variables, we restrict the factor to the observed values. For example, $f(x, y, e) := P(X = x \mid Y = y, E = e)$, and we have observed $E = e^*$, then we replace the factor f with $f'(x, y) := f(x, y, e^*)$.

2. Pointwise product and summing out Pick a hidden variable H to eliminate.

1. Identify all factors $f_1, f_2, \ldots, f_k$ that contain variable H.
2. Compute the pointwise product of these factors to get a new factor f.
3. Sum out variable H from factor f to get a new factor ϕ that does not contain H.
4. Add factor ϕ to the set of factors and remove $f_1, f_2, \ldots, f_k$.

3. Repeat Repeat the above step until a factor containing only the query variable X is obtained and no other factors contain X.

4. Normalization The remaining factor $\tau(X)$ gives an unnormalized distribution over X. Normalize it to get the final result:

$$P(X = x \mid \boldsymbol{E} = \boldsymbol{e}) = \frac{\tau(x)}{\sum_{x'} \tau(x')}$$

3.2.3 Time Complexity

Suppose:

- The BN has n nodes in total.
- Each random variable has at most d possible values.

A factor involving m variables has d^m entries.

The pointwise product of two factors with m_1 and m_2 variables results in a new factor with at most $m_1 + m_2$ variables, and takes $O(d^{m_1+m_2})$ time to compute.

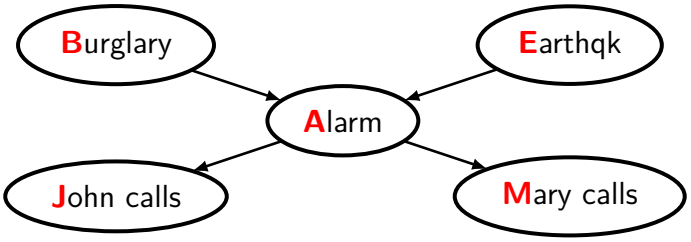
Summing out a variable from a factor with m variables results in a new factor with $m-1$ variables, and takes $O(d^m)$ time to compute.

The bottleneck occurs when doing pointwise products or summing out.

Conclusion The time complexity is $O(n \cdot d^{w+1})$, where w is the treewidth of the graph. In the worst case, w can be about n.

3.2.4 Another View of the Algorithm

We use an example to illustrate the process of variable elimination, to provide another perspective to understand the algorithm.



Given observations that $J = j$ and $M = m$, we want to compute the probability of burglary B. We start with the formula in the enumeration method:

$$
\begin{aligned}
P(B \mid j, m) &= \alpha \cdot \sum_e \sum_a P(B, e, a, j, m) \\
&= \alpha \cdot \sum_e \sum_a P(B) \cdot P(e) \cdot P(a \mid B, e) \cdot P(j \mid a) \cdot P(m \mid a) \qquad \text{by the Chain Rule} \\
&= \alpha \cdot \sum_e P(e) \cdot \sum_a P(a \mid B, e) \cdot P(j \mid a) \cdot P(m \mid a) \\
&\qquad P(B),\ P(e),\ P(a \mid B, e),\ P(j \mid a),\ P(m \mid a) \text{ correspond to 5 factors.} \\
&= \alpha \cdot P(B) \sum_e P(e) \cdot \sum_a f_1(B, e, a) \\
&\qquad 3 \text{ factors are merged. This is equivalent to performing pointwise products.} \\
&= \alpha \cdot P(B) \sum_e P(e) \cdot \phi_1(B, e) \qquad \text{Variable } A \text{ is summed out.}
\end{aligned}
$$

In the enumeration method, we do not factor out the common factors. In variable elimination, we extract common factors to reduce redundant computations.

But different order of eliminating variables leads to different sizes of intermediate factors, which may result in different time cost when performing pointwise products and summing out.

$$
\begin{aligned}
P(B \mid j, m) &= \alpha \cdot \sum_e \sum_a P(B) \cdot P(e) \cdot P(a \mid B, e) \cdot P(j \mid a) \cdot P(m \mid a) \\
&= \alpha \cdot P(B) \sum_a P(j \mid a) \cdot P(m \mid a) \sum_e P(e) \cdot P(a \mid B, e) \qquad \text{The summation order can be switched.}
\end{aligned}
$$

Finding the optimal order of eliminating variables is NP-hard. But there are some heuristics to find a good order that usually works well in practice. For example, the **min-fill** heuristic chooses the variable that minimizes the number of edges added to the graph when the variable is eliminated.

3.3 Rejection Sampling

Rejection sampling is a Monte Carlo method for approximate inference.

Main idea Simulate the generation of a large number of samples, reject those that do not match the observed Evidence, and finally calculate the frequency of the query variable appearing in the remaining samples.

3.3.1 Algorithm Steps

Assume we want to compute $P(X_Q \mid X_E = \tilde{e})$, where X_Q is the query variable and $X_E = \tilde{e}$ is the evidence.

1. Topological sort Obtain the sorted sequence $X_1, X_2, \ldots, X_n$, where every node is behind its parents.

2. Samples generation In each sample, nodes are instantiated in topological order. This guarantees that when generating X_i, its parents already hold concrete values, thereby determining the distribution of X_i via its CPT.

3. Consistency check (rejection step) If the value of the evidence variable X_E in the generated sample does not match the observed value, reject this sample (do not count it). Note: reject the whole sample (including the parents generated).

Suppose we generate N samples (including both accepted and rejected ones): $S^{(1)}, S^{(2)}, \ldots, S^{(N)}$, where each sample $S^{(i)}$ is a tuple of values: $S^{(i)} = (x_1^{(i)}, x_2^{(i)}, \ldots, x_n^{(i)})$. Suppose the query variable is X_Q. Then the query distribution can be approximated as:

$$
P(X_Q = \hat{x} \mid X_E = \tilde{e}) \ = \ \frac{\sum_{i=1}^{N} \left[x_Q^{(i)} = \hat{x}\right] \cdot \left[x_E^{(i)} = \tilde{e}\right]}{\sum_{i=1}^{N} \left[x_E^{(i)} = \tilde{e}\right]}
$$

where $\sum_{i=1}^{N} \left[x_E^{(i)} = \tilde{e}\right]$ is the number of accepted samples.

3.3.2 Problems

Low efficiency: when the evidence $X_E = e$ is a rare event, lots of samples are rejected, wasting a lot of time.

3.4 Likelihood Weighting

A directly improved version of rejection sampling.

Topological sort Same as in the rejection sampling.

Evidence instantiation Instead of randomly generating a value for the evidence variable E, we fix it to the observed value e, and continue to generate values for the following variables.

Weight The final sample is assigned a weight $w = P(E = e \mid \{\text{Parents}(E) = \text{generated values for the parents of } E\})$. By the definition, the weight can directly obtained from the CPT of E. If there are multiple evidence variables, the final weight is the product of the weights calculated for each evidence variable.

Suppose we generate N samples: $S^{(1)}, S^{(2)}, \ldots, S^{(N)}$, where each sample $S^{(i)}$ has a weight $w^{(i)}$ and is a tuple of values for all variables in the BN: $S^{(i)} = (x_1^{(i)}, x_2^{(i)}, \ldots, x_n^{(i)})$. Suppose the query variable is X_Q. Then the query distribution can be approximated as:

$$P(X_Q = \hat{x} \mid E = e) = \frac{\sum_{i=1}^{N} w^{(i)} \cdot [x_Q^{(i)} = \hat{x}]}{\sum_{i=1}^{N} w^{(i)}}$$

3.5 Gibbs Sampling

Gibbs sampling is a Markov Chain Monte Carlo (MCMC) method for approximate inference.

Main idea Different from rejection sampling and likelihood weighting, Gibbs sampling does not generate each sample independently. Instead, it starts from an initial state (a sample), and randomly walks through the state space (generating a sequence of samples, where each sample is generated based on the previous one).

3.5.1 Algorithm Steps

Suppose the BN has n variables.

1. Initial state Pick an initial state by fixing the evidence variables to their observed values, and randomly assigning values to all other variables. Let the initial state be $S_1 = (x_1^{(1)}, x_2^{(1)}, \ldots, x_n^{(1)})$.

2. Sample sequence generation In each iteration, we update the value of one non-evidence variable based on the current values of all other variables. Let the i-th state (sample) be $S_i = (x_1^{(i)}, x_2^{(i)}, \ldots, x_n^{(i)})$. To generate the next state S_{i+1}:

1. Pick a non-evidence variable. Suppose it is X_k.
2. Compute the conditional distribution of X_k given the current values of all other variables:

$$P\left(X_k \mid \{X_1 = x_1^{(i)}, \ldots, X_{k-1} = x_{k-1}^{(i)}, X_{k+1} = x_{k+1}^{(i)}, \ldots, X_n = x_n^{(i)}\}\right)$$

3. Sample a new value for X_k from the above distribution. Let the new value be $x_k^{(i+1)}$.
4. New state S_{i+1} is obtained by updating X_k:

$$S_{i+1} = (x_1^{(i)}, \ldots, x_{k-1}^{(i)}, x_k^{(i+1)}, x_{k+1}^{(i)}, \ldots, x_n^{(i)})$$

3. Derive the query distribution After generating a sufficiently large sample set, the conditional distribution of the query variable can be approximated by empirical frequency counts of its possible values.

3.5.2 Some Optimizations

Markov Blanket When computing the conditional distribution of a variable X_k, we do not need to consider all other variables. In Bayesian Networks, a variable is *conditionally independent* of all other variables given its **Markov Blanket**, which consists of:

- Its parents.
- Its children.
- The other parents of its children (co-parents).

Thus, we only need to compute:

$$
\begin{aligned}
&P\Big(X_k \mid \{\text{other variables} = \text{their previous values}\}\Big) \\
&\quad = P\Big(X_k \mid \{\text{MarkovBlanket}(X_k) = \text{their previous values}\}\Big) \\
&\quad \propto P\Big(X_k \mid \{\text{Parents}(X_k) = \text{their previous values}\}\Big) \cdot \prod_{C \in \text{Children}(X_k)} P\Big(C = \text{its previous value} \mid \{\text{Parents}(C) = \text{their previous values}\}\Big)
\end{aligned}
$$

Burn-in It takes some time for the Markov chain to converge to its stationary distribution. To reduce the influence of the initial state, we can discard the first several samples (for example, the first 1000 samples) and only use the later samples to estimate the query distribution. The discarded samples are called **burn-in** samples.

Thinning In the generated sample sequence, consecutive samples are highly correlated, which is against the assumption of *independent identically distributed (i.i.d.)* samples in statistical estimation. To reduce the correlation between samples, we can keep 1 sample every k samples (for example, every 100 samples) and discard the others.

3.5.3 Pros and Cons

Pros Usually more efficient than likelihood weighting, especially in the presence of strong evidence in downstream variables.

Cons Can be slow to converge, especially when there exist multiple separated high-probability regions in the state space.